PROJECT REPORT FOR

Foundations of Artificial Intelligence

AI-BASED EXAM DUTY AND LABORATORY ALLOCATION SYSTEM USING CONSTRAINT SATISFACTION AND SEARCH ALGORITHMS

A PROJECT REPORT

Submitted by

2116240701235 – KARTHIKEYAN A

2116240701221 – KAARTHIKA



October 2026

BONAFIDE CERTIFICATE

Certified that this project report "AI-BASED EXAM DUTY AND LABORATORY ALLOCATION SYSTEM USING CONSTRAINT SATISFACTION AND SEARCH ALGORITHMS" is the bonafide work of "KARTHIKEYAN A (2116240701235) and KAARTHIKA (2116240701221)" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY IN-CHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
	1.1 Background	
	1.2 Problem statement	
	1.3 Objectives	
	1.4 Scope	
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	9
	3.2 Architecture diagram	
	3.3 CSP formulation	
	3.4 Uniqueness of the proposed approach	
4	MODULES DESCRIPTION	12
	4.1 Data management module	
	4.2 Constraint management module	
	4.3 CSP allocation engine	
	4.4 Incident-driven rescheduling and explanation module	
5	ALGORITHMS	15
	5.1 AC-3 constraint propagation	
	5.2 Backtracking search with heuristics	
	5.3 Min-conflicts repair with change penalty	
	5.4 Hill climbing for soft constraints	
6	IMPLEMENTATION AND RESULTS	18
	6.1 Technology stack	
	6.2 Dataset	
	6.3 Experimental setup	
	6.4 Expected results and evaluation metrics	
7	CONCLUSION AND FUTURE WORK	21
	REFERENCES	22

ABSTRACT

Allocating examination venues and invigilation duties is a recurring scheduling task in every academic institution. The examination cell must ensure that each examination is held in a room of adequate capacity, that laboratory examinations are placed in laboratories with enough working systems, and that every invigilator is free of lectures and other duties in the assigned slot while duty loads remain balanced across faculty. The task is usually performed manually in spreadsheets, is error-prone, and must be repeated whenever a laboratory becomes unavailable, an examination is postponed, or a faculty member reports leave.

This project formulates the problem as a Constraint Satisfaction Problem (CSP). Each examination session is a variable whose value is a (venue, invigilators) pair; domains are built from room capacity, system availability and faculty timetables; institutional rules are expressed as seven hard constraints and five weighted soft constraints. The initial allocation is generated using AC-3 constraint propagation followed by backtracking search with the Minimum Remaining Values, degree and Least Constraining Value heuristics and forward checking, and refined by hill climbing on the soft-constraint cost.

The distinguishing feature of the system is incident-driven rescheduling. When a disruption is reported, only the affected assignments are released and repaired using min-conflicts local search with a change penalty, so that the new allocation differs from the previous one in as few assignments as possible. If no repair exists, an explanation module identifies the blocking constraint and proposes relaxations. An independent validator checks every published allocation against all hard constraints. The solver is exposed through a role-based web application with college-domain login, incident reporting, duty charts and notifications. Evaluation is planned on synthetic instances of up to 150 examinations using hard-constraint violations, runtime, backtracks, number of changed assignments per incident and duty-load fairness as metrics.

Keywords: Constraint satisfaction problem, AC-3, backtracking search, MRV heuristic, forward checking, min-conflicts, local search, examination scheduling, rescheduling.

CHAPTER 1 INTRODUCTION

1.1 Background

Examinations are conducted several times each semester, and for each examination session the examination cell must decide where the examination is held and which faculty members invigilate it. Theory examinations need halls whose seating capacity is at least the number of candidates. Laboratory examinations need laboratories with a sufficient number of working systems and the required software. Every invigilator must be free in that slot: not lecturing, not invigilating elsewhere, not on leave, and not the instructor of the course being examined. Departmental norms further limit the number of duties a person may take per day and expect the total duty load to be spread evenly across staff.

In most institutions this allocation is prepared manually. The number of combinations grows quickly: a session of forty examinations across twenty-five rooms and eighty faculty members has far more candidate assignments than can be checked by hand, and a single oversight produces a double-booked room or a faculty member expected in two places at once. The problem is compounded by change. Laboratories go down, systems fail, examinations are postponed and faculty members fall ill, and each such event forces the allocation to be revised under time pressure, often on the morning of the examination.

1.2 Problem statement

Given an examination timetable, an inventory of rooms and laboratories, and the teaching timetables, leave records and preferences of faculty members, the system must (i) assign to every examination session a venue and the required number of invigilators such that all institutional hard constraints are satisfied and the weighted soft-constraint cost is minimised; and (ii) when an incident is reported after publication, produce a new valid allocation that minimises the number of assignments changed with respect to the published one, or, when no such allocation exists, identify the constraint responsible and suggest relaxations.

1.3 Objectives

1. To formulate exam duty and laboratory allocation as a Constraint Satisfaction Problem with explicitly enumerated hard constraints (H1–H7) and weighted soft constraints (S1–S5).
2. To implement AC-3 constraint propagation and backtracking search with the MRV, degree and LCV heuristics and forward checking to generate a valid initial allocation.

3. To implement min-conflicts local search with a change penalty for incident-driven repair of an existing allocation.
4. To build an independent validator and an infeasibility explanation module.
5. To deliver the solver through a role-based web application with college-domain authentication, incident reporting, duty charts and notifications.
6. To evaluate the system on synthetic instances using hard-constraint violations, runtime, number of backtracks, assignments changed per incident and duty-load fairness.

1.4 Scope

The examination timetable (which course is examined on which date and slot) and the regular class timetable are treated as inputs. The system does not generate examination dates; it assigns venues and invigilators to sessions whose dates are already fixed, and changes a date only when an incident explicitly postpones an examination. Student clash-freeness is therefore the responsibility of the timetable supplied as input. This boundary keeps the problem tractable within a semester and keeps the focus on the search algorithms.

CHAPTER 2 LITERATURE REVIEW

Educational timetabling has been studied for several decades and is a standard benchmark domain for constraint satisfaction and combinatorial optimisation. The work most relevant to this project falls into three groups: constraint satisfaction foundations, examination timetabling surveys, and work on rescheduling under disruption.

2.1 Constraint satisfaction foundations

Mackworth [2] introduced the notion of arc consistency and the AC-3 algorithm, which removes from each variable's domain every value that has no supporting value in a neighbouring variable's domain. AC-3 remains the standard pre-processing step for CSPs because it is simple, terminates quickly and can detect infeasibility before any search is performed. Russell and Norvig [1] present the complete CSP toolkit used in this project: backtracking search, the Minimum Remaining Values (MRV) and degree heuristics for variable ordering, the Least Constraining Value (LCV) heuristic for value ordering, forward checking for early failure detection, and the min-conflicts local search algorithm. The same text notes that min-conflicts is particularly effective when a nearly correct assignment is already available, which is exactly the situation after an incident in a published schedule.

2.2 Examination timetabling

Qu, Burke, McCollum, Merlot and Lee [4] survey search methodologies for examination timetabling, covering graph-colouring heuristics, constraint-based methods, metaheuristics such as tabu search and simulated annealing, and hyper-heuristics. Siew et al. [3] provide a recent review of solution methodologies and the constraint sets used in benchmark instances such as the Toronto and ITC 2007 datasets. Tan et al. [5] and the review in [7] extend this to school and university timetabling, cataloguing hard and soft constraints and open issues. Mamay [6] surveys optimisation approaches to examination scheduling with emphasis on practical deployment.

Across these surveys the problem addressed is the generation of the examination timetable itself: assigning examinations to periods and rooms so that no student has clashing examinations. Invigilator allocation is treated, when at all, as a post-processing step, and faculty teaching timetables are rarely modelled as constraints. The methods are also designed to produce a schedule from scratch and are evaluated on solution quality rather than on how much a schedule changes when it must be revised.

2.3 Rescheduling under disruption

Rescheduling with minimal perturbation is recognised as an open problem in the surveys above, and has been studied mainly in production scheduling and airline crew recovery, where the cost of disturbing an already communicated plan is explicit. The common finding is that local search seeded from the existing solution outperforms re-optimisation from scratch when the objective includes a stability term. Applying this idea to invigilator duties, where every changed assignment means a faculty member must be notified and may object, is the gap this project addresses.

2.4 Summary of the gap

- Existing systems generate a schedule once; after a disruption they re-solve entirely and may move assignments unrelated to the disruption.
- Invigilator duties with faculty teaching-timetable and workload constraints are rarely modelled together with venue allocation.
- Infeasible cases are reported as failures without identifying the responsible constraint.

The proposed system addresses these three gaps with one CSP model solved by two complementary search algorithms, and an explanation module for infeasible cases.

CHAPTER 3 PROPOSED SYSTEM

3.1 Overview

The system has two operating modes. In generation mode the administrator imports the examination list, room inventory and faculty data, and the CSP engine produces a complete allocation of venues and invigilators that satisfies every hard constraint and has low soft-constraint cost. In repair mode, which is triggered by an incident report from a faculty member, laboratory staff or the administrator, the engine releases only the assignments affected by the incident, searches for replacement values that satisfy all hard constraints, and prefers replacements that leave the rest of the published allocation untouched. In both modes the result passes through an independent validator before it is stored and published, and in both modes the system records why each assignment was chosen so that the administrator can inspect the reasoning.

3.2 Architecture diagram

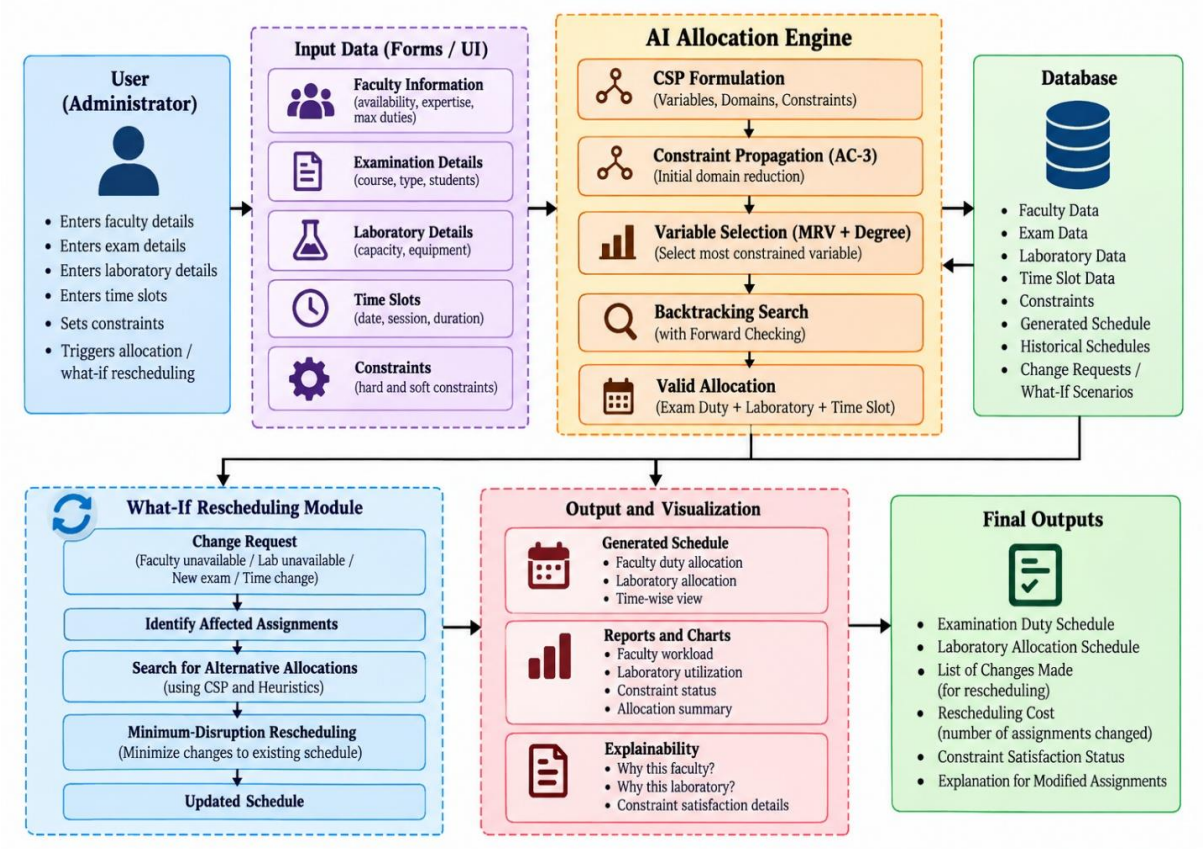


Figure 3.1 System architecture

The administrator supplies faculty, examination, laboratory, time-slot and constraint data through the web interface; the data is stored in a relational database. The CSP allocation engine

builds variables and domains from the stored data, applies AC-3 to prune the domains, and runs backtracking search with the MRV and degree heuristics and forward checking to obtain a valid allocation, which is then refined by hill climbing on the soft-constraint cost. The incident-driven rescheduling module receives change requests, identifies the affected assignments, repairs them with min-conflicts and reports the list of changes and the rescheduling cost. The output and visualisation layer presents the schedule, workload and utilisation charts, constraint status and the explanation for every assignment, and the final outputs are the examination duty schedule, laboratory allocation, change list, rescheduling cost, constraint-satisfaction status and explanations.

3.3 CSP formulation

3.3.1 Variables

For every examination session E_i (a course examined on a given date and slot) there are two variables: $room_i$, the venue, and $staff_i$, the set of invigilators. Laboratory sessions that are not examinations but occupy laboratories (regular practical classes) are fixed assignments that reduce room availability and are not variables.

3.3.2 Domains

The domain of $room_i$ is the set of rooms whose capacity is at least the strength of E_i, whose type matches (theory hall or laboratory), which for laboratory examinations have at least the required number of working systems and the required software, and which are marked available in that slot. The domain of $staff_i$ is the set of faculty members who have no lecture, no other duty and no leave in that slot and who are not the instructor of the examined course. The required number of invigilators k_i is $\lceil strength_i / 30 \rceil$.

3.3.3 Constraints

Table 3.1 lists the hard constraints, which must all hold in any published allocation, and Table 3.2 the soft constraints, which are combined into a weighted cost that the solver minimises.

ID	Hard constraint	Type
H1	At most one examination per room per slot	Binary (between exams)
H2	Room capacity ≥ strength; for laboratories, working systems ≥ strength	Unary
H3	Room status is available in that slot (no booking, not reported down)	Unary

ID	Hard constraint	Type
H4	Each invigilator is free in that slot: no lecture, no other duty, no leave	Binary
H5	Invigilator is not the instructor of the examined course	Unary
H6	At most two duties per faculty member per day	Global
H7	Number of invigilators assigned equals ⌈ strength / 30⌉	Unary

Table 3.1 Hard constraints

ID	Soft constraint	Cost term
S1	Balanced duty count across faculty	Variance of duties per faculty
S2	Invigilator's department block near the venue	Count of cross-block assignments
S3	No back-to-back duties for a faculty member	Count of consecutive-slot duties
S4	Faculty slot preferences respected	Count of disliked slots assigned
S5	Minimal change from the previously published allocation (repair mode only)	Number of assignments that differ

Table 3.2 Soft constraints

3.3.4 Objective

The solver minimises $\text{cost}(A) = \Sigma_k w_k \cdot S_k(A) + \lambda \cdot \text{changes}(A, A_{prev})$ subject to H1–H7, where w_k are administrator-set weights, λ is zero during initial generation and large during repair, and $\text{changes}(A, A_{prev})$ counts the assignments in A that differ from the previously published allocation. The λ term is what makes repair prefer local fixes over global re-arrangement.

3.4 Uniqueness of the proposed approach

Standard timetabling systems treat the schedule as disposable: after any change the solver runs again and whatever it produces is published. The proposed system treats the published schedule as a commitment made to faculty and students. It uses a complete search algorithm (backtracking) only when there is no schedule to preserve, and an incomplete but fast local search (min-conflicts) seeded from the existing schedule when there is. The success of a repair is measured by how few people are affected, not only by whether the result is valid. In addition, when no repair exists, the system reports which constraint blocked it and what relaxation (a different date, a larger room, a third duty for one faculty member) would make it feasible, turning a failure into a decision for the administrator.

CHAPTER 4 MODULES DESCRIPTION

The system is organised into five modules. Modules 1 and 2 manage data and rules, Module 3 is the generation engine, Module 4 is the repair and explanation engine, and Module 5 is the user-facing application. Modules 3 and 4 form a standalone Python package with no dependency on the web layer, so they can be exercised from the command line and tested independently.

Module	Responsibility
1 Data management	Import and maintain examinations, rooms and laboratories, faculty, teaching timetables, leave and existing laboratory bookings.
2 Constraint management	Register hard and soft constraints as individual checker functions with weights; expose weights to the administrator.
3 CSP allocation engine	Build domains, run AC-3, backtracking with MRV/degree/LCV and forward checking, hill climbing on soft cost.
4 Rescheduling and explanation	Accept incidents, identify affected assignments, run min-conflicts repair with change penalty, validate, produce diff, explain infeasibility.
5 Web application	Role-based pages for administrator, head of department and faculty; college-domain login; incident form; duty charts; notifications.

Table 4.1 Module summary

4.1 Data management module

This module imports four CSV files and validates them on import. exams.csv holds course code, department, date, slot, strength, type (theory or laboratory), required software and instructor. rooms.csv holds room identifier, block, type, capacity, number of systems, installed software and status. faculty.csv holds identifier, department, block, maximum duties per day and slot preferences. timetable.csv holds every lecture and regular laboratory session as (faculty or room, day, slot) tuples, plus leave records. From these the module builds two availability matrices, room × slot and faculty × slot, which are the raw material for the domains in Module 3. Import errors such as duplicate identifiers, missing capacity or an instructor not present in the faculty list are reported to the administrator before any solving takes place.

4.2 Constraint management module

Each constraint in Tables 3.1 and 3.2 is implemented as a small Python function that receives the current partial assignment and the problem data and returns either a boolean (hard) or a numeric penalty (soft). The functions are registered in a list with their identifier, kind and weight. The solver iterates over this list rather than containing constraint logic itself, so adding

an institutional rule means adding one function. Soft-constraint weights are editable by the administrator from the web interface, which allows the trade-off between fairness and proximity, for example, to be tuned per examination session.

4.3 CSP allocation engine

This module is the main artificial-intelligence component. It constructs the variables and domains described in Section 3.3, runs AC-3 over all binary arcs (pairs of sessions in the same slot for H1, and pairs sharing a candidate invigilator for H4), and reports infeasibility immediately if any domain becomes empty. Backtracking search then assigns sessions one at a time, choosing the session with the smallest remaining domain (MRV), breaking ties by the session involved in the most constraints (degree), and trying room and staff values in an order that rules out the fewest options for other sessions (LCV). After each assignment forward checking removes the chosen room and staff from the domains of all sessions in the same slot and backtracks as soon as any domain empties. The engine records the number of nodes expanded and backtracks performed so that the effect of the heuristics can be measured. Once a hard-feasible allocation is found, hill climbing with random restarts performs swap moves that reduce the soft-constraint cost while preserving hard feasibility.

4.4 Incident-driven rescheduling and explanation module

An incident is one of four types: room or laboratory unavailable for a date range, systems in a laboratory reduced to a given working count, examination postponed to a new date and slot, or faculty member unavailable for a date range. The module computes the set of assignments invalidated by the incident, releases only those variables, rebuilds their domains from the updated availability matrices, and runs min-conflicts local search. At each step min-conflicts selects a conflicted variable and assigns it the value that minimises the number of hard-constraint violations plus λ for any value that differs from the previously published one. When the validator reports zero violations the repaired allocation is stored as a new version and a diff listing every changed assignment is produced. If min-conflicts does not converge within the step limit, backtracking is run on the released variables alone; if that also fails, the explanation sub-module reports, for each unassignable session, which hard constraint emptied its domain and which relaxations would restore candidates. Every incident, repair and explanation is written to an audit log.

4.5 Web application and notification module

The application has three roles. The administrator imports data, sets constraint weights, triggers generation, views the allocation grid, reviews the incident queue with a preview of each proposed repair, and approves or rejects it. Heads of department see their faculty's duties and approve swap requests. Faculty members see their own duties, download a calendar file, report incidents and request swaps. Authentication is restricted to the college e-mail domain; a roster-based login is used for the first prototype and Google OAuth with hosted-domain restriction is planned for deployment. After an approved repair only the faculty members whose assignments changed are notified by e-mail with the before and after values.

CHAPTER 5 ALGORITHMS

This chapter presents the algorithms implemented in Modules 3 and 4. All four are implemented from first principles in Python; no external constraint solver is used in the core.

5.1 AC-3 constraint propagation

AC-3 enforces arc consistency: for every arc (X_i, X_j) it removes from the domain of X_i every value that has no compatible value in the domain of X_j, and re-queues every arc into X_i whenever its domain shrinks. In this project the arcs are between the room variables of sessions in the same slot (constraint H1) and between the staff variables of sessions in the same slot (constraint H4). Running AC-3 before search shrinks domains and detects some infeasible instances outright.

```
function AC3(csp):
    queue ← all arcs (Xi, Xj) in csp
    while queue not empty:
        (Xi, Xj) ← queue.pop()
        if REVISE(csp, Xi, Xj):
            if D(Xi) is empty: return false      // infeasible
            for each Xk in neighbours(Xi) \ {Xj}:
                queue.push((Xk, Xi))
    return true

function REVISE(csp, Xi, Xj):
    revised ← false
    for each x in D(Xi):
        if no y in D(Xj) satisfies constraint(Xi=x, Xj=y):
            remove x from D(Xi); revised ← true
    return revised
```

Worked example. Sessions E1 (120 students) and E2 (60 students) share a slot. After unary filtering, D($room_1$) = {Hall-A} and D($room_2$) = {Hall-A, Hall-B}. Revising the arc ($room_2$, $room_1$) removes Hall-A from D($room_2$), because choosing it would leave $room_1$ with no compatible value. The search therefore starts with $room_2$ already fixed to Hall-B.

5.2 Backtracking search with heuristics

Backtracking assigns one variable at a time and undoes the assignment when a dead end is reached. Three heuristics and one inference step make it practical. MRV selects the variable with the fewest remaining legal values so that failures are discovered early. The degree heuristic breaks MRV ties in favour of the variable that participates in the most constraints with unassigned variables. LCV orders the values of the chosen variable so that the value which eliminates the fewest options for neighbouring variables is tried first. Forward checking,

performed after every assignment, deletes the chosen value from the domains of all neighbours and triggers backtracking as soon as any domain becomes empty.

```
function BACKTRACK(assignment, csp):
    if assignment is complete: return assignment
    X ← SELECT-UNASSIGNED(csp)            // MRV, ties by degree
    for each v in ORDER-VALUES(X, csp):   // LCV
        if v is consistent with assignment under H1..H7:
            assignment[X] ← v
            saved ← FORWARD-CHECK(X, v, csp)   // prune neighbours
            if no domain is empty:
                result ← BACKTRACK(assignment, csp)
                if result ≠ failure: return result
            restore domains from saved
            delete assignment[X]
    return failure
```

Each session contributes two variables, $room_i$ and $staff_i$. They are assigned together as a pair so that the staff domain, which depends on the slot and not on the room, is filtered once per session. The engine counts nodes expanded and backtracks, which are reported as metrics in Chapter 6.

5.3 Min-conflicts repair with change penalty

Min-conflicts is a local search algorithm for CSPs. It starts from a complete (possibly inconsistent) assignment, repeatedly selects a variable that participates in a violated constraint, and reassigns it the value that minimises the number of violated constraints. Its strength is that, when the starting assignment is almost correct, it converges in a handful of steps. In this project the starting assignment is the published allocation with the incident applied, which is exactly such a case. The standard algorithm is extended with a change penalty λ so that, among values with equal conflict counts, the value already published is preferred, and values that disturb unaffected sessions are avoided.

```
function MIN-CONFLICTS-REPAIR(published, incident, lambda, max_steps):
    current ← published
    affected ← assignments invalidated by incident
    for each X in affected: current[X] ← undefined; rebuild D(X)
    for step = 1 .. max_steps:
        if VALIDATE(current) reports no violations: return current
        X ← random conflicted or undefined variable
        v ← argmin over D(X) of
              CONFLICTS(X, v, current) + lambda * [v ≠ published[X]]
        current[X] ← v
    return BACKTRACK on affected variables only   // complete fallback
```

The function CONFLICTS counts hard-constraint violations that assigning v to X would create. Because λ is added only when v differs from the published value, assignments outside the

affected set are changed only when every alternative for an affected session conflicts, which is the behaviour an examination cell expects.

5.4 Hill climbing for soft constraints

Once a hard-feasible allocation exists, hill climbing improves its soft-constraint cost. A move swaps the invigilators of two sessions or moves a session to another free room of adequate capacity; a move is accepted if it preserves hard feasibility and lowers cost(A). The search stops when no improving move exists, and is restarted from a random feasible neighbour a fixed number of times to escape local minima. The same routine runs after a repair with λ included in the cost, so that soft improvements never undo the stability achieved by min-conflicts.

5.5 Validator and infeasibility explanation

The validator is a separate module that imports no solver code. It receives an allocation and the problem data and checks every hard constraint from first principles, returning a list of violations with the constraint identifier and the sessions involved. Every allocation, whether generated or repaired, must pass the validator before it is stored. The test suite asserts on the validator's output rather than on the solver's own success flag. The explanation routine is invoked when a session cannot be assigned. It replays unary filtering and AC-3 for that session alone, records which constraint removed the last candidate from each domain, and then tests a fixed list of relaxations — moving the session to an adjacent slot or date, allowing a room of the next capacity class, or permitting one faculty member a third duty — reporting those that restore at least one candidate.

CHAPTER 6 IMPLEMENTATION AND RESULTS

At the time of this report the data model, CSV import, synthetic data generator and validator are implemented and the CSP engine is in progress. This chapter describes the implementation environment, the dataset, the experiments planned for the first review and the metrics by which the system will be judged.

6.1 Technology stack

Layer	Technology	Purpose
Language	Python 3.12	CSP engine, validator, data tools
CSP engine	Own package: ac3, backtracking, local_search, cost, validator, explain	All algorithms implemented from the syllabus; no external solver
API	FastAPI, Pydantic	Endpoints for generate, incident, allocation, explain; background solver jobs
Database	PostgreSQL (SQLite during development), SQLAlchemy	Versioned allocations, incidents, audit log
Frontend	React with Vite	Allocation grid, incident form, repair diff view
Authentication	Roster-based login; Google OAuth with hosted-domain restriction planned	Access limited to college e-mail accounts
Notifications	SMTP e-mail, ICS calendar export	Inform only affected faculty after a repair
Testing	pytest, Hypothesis	Unit, scenario and property-based tests
Deployment	Docker; Render (API, database) and Vercel (frontend)	Public demo with an offline fallback

Table 6.1 Technology stack

6.2 Dataset

No public dataset matches the invigilator-allocation problem with faculty teaching timetables, so a synthetic generator was written. It takes the number of examinations, rooms, faculty members, days and slots per day, together with a disruption rate, and produces the four input files with realistic distributions: class strengths between 30 and 120, room capacities drawn from the set {40, 60, 90, 120}, laboratories with 30 to 60 systems and a small fraction of non-working ones, and faculty teaching loads of 12 to 16 hours per week. Three instance sizes are used: a small instance of 10 examinations for worked examples, the target instance of 40 examinations, 25 rooms, 80 faculty members and 12 slots, and a stress instance of 150

examinations for scaling experiments. An anonymised timetable from the college examination cell will be used for the final demonstration.

6.3 Experimental setup

All experiments run on a standard laptop (8 GB RAM) with the engine invoked from the command line so that timing excludes the web layer. Each configuration is run ten times with different random seeds and the mean is reported. The following experiments are planned.

1. Heuristic ablation: backtracking with and without MRV, LCV and forward checking on all three instance sizes, measuring runtime and nodes expanded.
2. Effect of AC-3: domain sizes and infeasibility detections before and after propagation.
3. Repair versus re-solve: for each of four incident types applied to the target instance, number of changed assignments and runtime using min-conflicts repair versus full regeneration.
4. Fairness: standard deviation of duties per faculty member with and without the hill-climbing stage.
5. Infeasibility explanation: injected cases with no feasible repair, checking that the blocking constraint reported is the one that was injected.

6.4 Expected results and evaluation metrics

Metric	Definition	Target
Hard violations	Count returned by the validator on the published allocation	0 in every case
Generation runtime	Wall time for AC-3 + backtracking + hill climbing	< 5 s on the target instance
Nodes / backtracks	Search effort with and without heuristics	Order-of-magnitude reduction with MRV + FC
Changes per incident	Assignments that differ from the published allocation after repair	≤ 5 with repair; substantially fewer than full re-solve
Repair runtime	Wall time from incident to validated repair	< 2 s
Duty fairness	Standard deviation of duties per faculty member	Reduced by hill climbing
Explanation accuracy	Fraction of injected infeasible cases where the reported constraint matches	100 % on injected cases

Table 6.2 Evaluation metrics

Results will be presented as a schedule table, a faculty-workload chart, a room-utilisation chart, a constraint-status summary, a before-and-after comparison for each incident, and two plots: search effort versus instance size for each heuristic configuration, and changed assignments per incident for repair versus re-solve. The second plot is the principal evidence for the project's contribution.

6.5 Testing strategy

- Unit tests: one test per constraint function, with a passing and a failing assignment each.
- Validator property test: for randomly generated allocations with an injected violation, the validator must report that violation (Hypothesis-based).
- Scenario tests: laboratory down, systems reduced, examination postponed, faculty unavailable, and no room free; each asserts validator output and the size of the diff.
- Minimal-change test: on the target instance, repair must change no more assignments than full regeneration for every incident type.
- Authentication tests: non-college e-mail rejected; faculty cannot reach administrator pages.

CHAPTER 7 CONCLUSION AND FUTURE WORK

This project applies constraint satisfaction and search techniques from the Foundations of Artificial Intelligence syllabus to a scheduling task that every institution performs by hand. The allocation of venues and invigilators is modelled as a CSP with explicit hard constraints H1–H7 and weighted soft constraints S1–S5. AC-3 propagation and backtracking search with the MRV, degree and LCV heuristics and forward checking generate a valid initial allocation, and hill climbing improves its fairness and convenience. The contribution beyond standard timetabling is the treatment of the published schedule as something to protect: incidents are repaired with min-conflicts local search under a change penalty so that as few faculty members as possible are disturbed, and infeasible cases are explained in terms of the blocking constraint and the relaxations that would resolve it. An independent validator guarantees that nothing invalid is published.

For the first review the group will demonstrate the engine from the command line on the synthetic target instance: generation with search statistics, one incident of each type repaired with a diff, and one injected infeasible case with its explanation. The web application, authentication and notifications follow once the engine is stable.

Several extensions are possible. The examination timetable itself could be generated by the same engine, adding student-clash constraints and increasing the problem size. Constraint weights could be learned from the administrator's manual overrides. A per-room reliability score updated from incident history would let the solver prefer dependable laboratories, introducing a small probabilistic component. Finally, benchmarking the hand-written engine against a commercial constraint solver would quantify the cost of implementing the algorithms from first principles.

REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ: Pearson, 2021, ch. 4 and 6.

[2] A. K. Mackworth, “Consistency in networks of relations,” Artificial Intelligence, vol. 8, no. 1, pp. 99–118, 1977.

[3] E. S. K. Siew, S. N. Sze, S. L. Goh, G. Kendall, N. R. Sabar, and S. Abdullah, “A survey of solution methodologies for exam timetabling problems,” IEEE Access, vol. 12, pp. 41479–41498, 2024.

[4] R. Qu, E. K. Burke, B. McCollum, L. T. G. Merlot, and S. Y. Lee, “A survey of search methodologies and automated system development for examination timetabling,” Journal of Scheduling, vol. 12, no. 1, pp. 55–89, 2009.

[5] J. S. Tan, S. L. Goh, G. Kendall, and N. R. Sabar, “A survey of the state-of-the-art of optimisation methodologies in school timetabling problems,” Expert Systems with Applications, vol. 165, art. 113943, 2021.

[6] K. Mamay, “Optimizing exam schedules: a literature survey,” Journal of Emerging Technologies and Computing, vol. 62, no. 1, pp. 65–74, 2023.

[7] A. Abuhamdah et al., “Optimization techniques in university timetabling problem: constraints, methodologies, benchmarks, and open issues,” Computers, Materials & Continua, vol. 74, no. 3, pp. 6461–6484, 2022.

[8] S. Minton, M. D. Johnston, A. B. Philips, and P. Laird, “Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems,” Artificial Intelligence, vol. 58, no. 1–3, pp. 161–205, 1992.

[9] B. McCollum et al., “Setting the research agenda in automated timetabling: the second international timetabling competition,” INFORMS Journal on Computing, vol. 22, no. 1, pp. 120–130, 2010.